Music Playlist Management System (Linked List)

CCE-2 · Unit-3 · C++ Lab Report

1. Title

Music Playlist Management System using a Doubly Linked List in C++

2. Problem Statement

Design and implement a console-based Music Playlist Management System in C++ using a doubly linked list. Each node represents one song and stores the song name, artist and duration. The user can add songs, delete songs, search for songs, display the playlist, and move to the next or previous song.

3. Objectives

- To implement a real-life application using a linked list in C++.
- To understand dynamic memory allocation (`new` / `delete`) and pointers.
- To perform insertion, deletion, searching and traversal on a linked list.
- To implement a menu-driven program using loops and a `switch` case.
- To understand how a doubly linked list allows movement in both directions (Next / Previous).

4. Data Structure Used

Each song is a node of a doubly linked list.

Field	Type	Purpose
`name`	string	Song name
`artist`	string	Artist name
`duration`	int	Length in seconds (shown as mm:ss)

Field	Type	Purpose
`next`	Song*	Pointer to the following song
`prev`	Song*	Pointer to the previous song

Three global pointers manage the list: `head` (first song), `tail` (last song) and `current` (song being played). All start as `NULL`.

5. Algorithm

Step 1: Start the program.

Step 2: Create the node structure with name, artist, duration, next and prev. Initialize head, tail and current as NULL.

Step 3: Display the menu: 1. Add, 2. Delete, 3. Search, 4. Display, 5. Next, 6. Previous, 7. Exit.

Step 4: Read the user's choice.

Step 5 (Add Song): Create a new node and read the song details. If the list is empty, make it head, tail and current. Otherwise attach it after tail and update tail.

Step 6 (Delete Song): Read the song name and traverse the list to find it. If not found, display "Song not found". If found, link its previous and next nodes to each other (update head / tail if needed), move current if it was deleted, then free the node.

Step 7 (Search Song): Read the song name and traverse from head. Display the details if found, otherwise display "Song not found".

Step 8 (Display): If the list is empty, display "Playlist is empty". Otherwise traverse from head to the end and print every song.

Step 9 (Next Song): If current → next is not NULL, move current forward and display it. Otherwise display "End of Playlist".

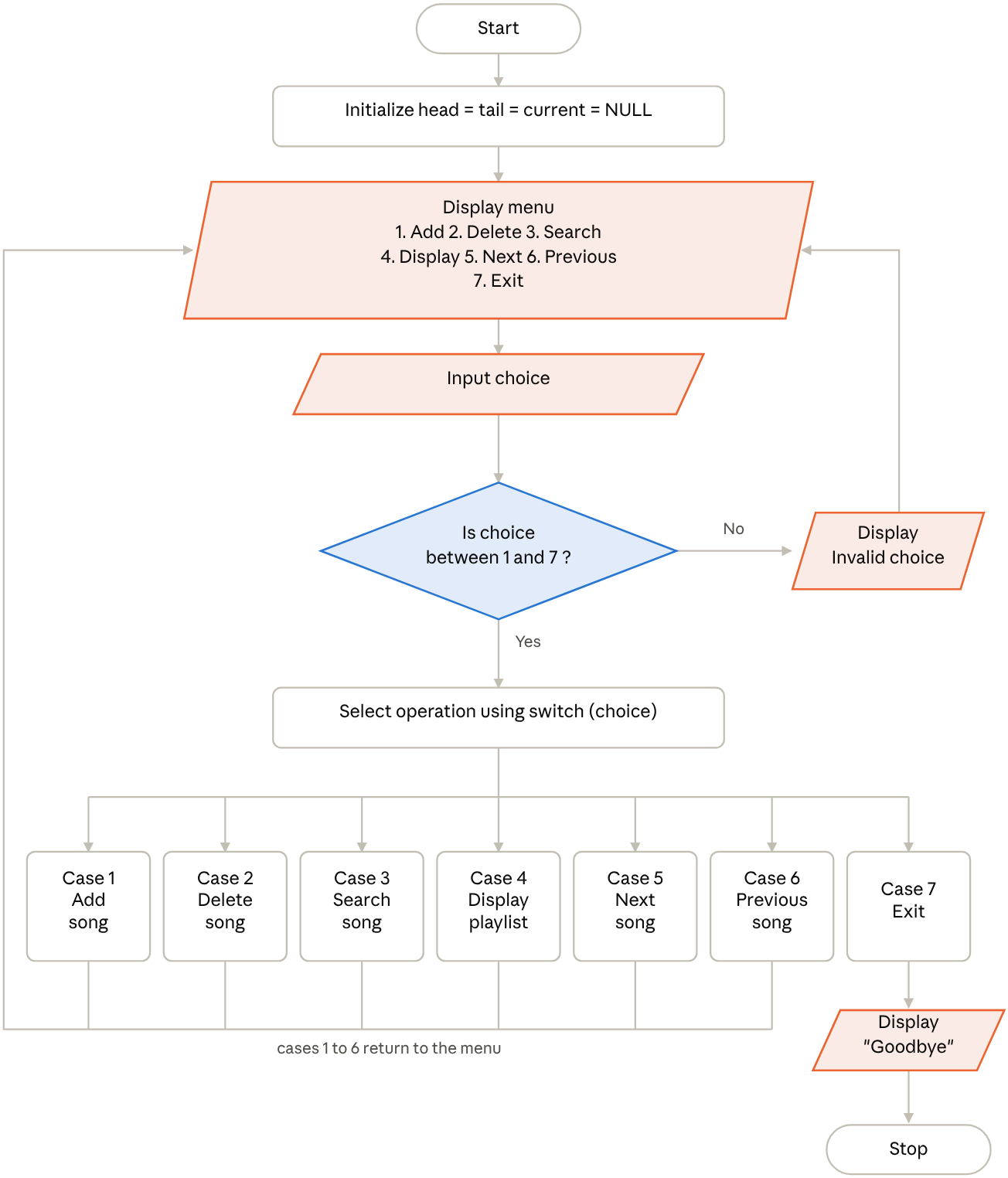
Step 10 (Previous Song): If current → prev is not NULL, move current back and display it. Otherwise display "Start of Playlist".

Step 11: Repeat Steps 3 to 10 until the user chooses Exit.

Step 12: Stop the program.

6. Flowchart

(See flowchart below.)



playlist menu flow · cases 1 to 6 loop back, case 7 exits

Invalid choices and cases 1 to 6 return to the menu; only case 7 prints "Goodbye" and stops.

7. C++ Program

```
#include <iostream>
#include <string>
using namespace std;

struct Song
{
    string name, artist;
    int duration;              // duration in seconds
    Song *next, *prev;
};

Song *head = NULL, *tail = NULL, *current = NULL;

void showSong(Song *s)
{
    int sec = s->duration % 60;
    cout << s->name << " | " << s->artist << " | "
         << s->duration / 60 << ":" << (sec < 10 ? "0" : "") << sec << endl;
}

// Add song at the end
void addSong()
{
    Song *s = new Song;
    cin.ignore();
    cout << "Enter song name: ";
    getline(cin, s->name);
    cout << "Enter artist name: ";
    getline(cin, s->artist);
    cout << "Enter duration (in seconds): ";
    cin >> s->duration;
    s->next = NULL;
    s->prev = NULL;

    if (head == NULL)
        head = tail = current = s;
    else
```

```
    {
        tail->next = s;
        s->prev = tail;
        tail = s;
    }
    cout << "Song added!\n";
}

// Delete song by name
void deleteSong()
{
    string n;
    cin.ignore();
    cout << "Enter song name to delete: ";
    getline(cin, n);

    Song *t = head;
    while (t != NULL && t->name != n)
        t = t->next;

    if (t == NULL)
    {
        cout << "Song not found!\n";
        return;
    }
    if (t->prev != NULL) t->prev->next = t->next;
    else head = t->next;

    if (t->next != NULL) t->next->prev = t->prev;
    else tail = t->prev;

    if (current == t)
        current = (t->next != NULL) ? t->next : t->prev;
    delete t;
    cout << "Song deleted!\n";
}

// Search song by name
void searchSong()
{
    string n;
    cin.ignore();
    cout << "Enter song name to search: ";
```

```
    getline(cin, n);

    Song *t = head;
    while (t != NULL)
    {
        if (t->name == n)
        {
            cout << "Song found: ";
            showSong(t);
            return;
        }
        t = t->next;
    }
    cout << "Song not found!\n";
}

// Display playlist
void displayPlaylist()
{
    if (head == NULL)
    {
        cout << "Playlist is empty!\n";
        return;
    }
    int i = 1;
    for (Song *t = head; t != NULL; t = t->next)
    {
        cout << i++ << ". ";
        showSong(t);
    }
}

void nextSong()
{
    if (current == NULL)
        cout << "Playlist is empty!\n";
    else if (current->next == NULL)
        cout << "End of Playlist!\n";
    else
    {
        current = current->next;
        cout << "Now playing: ";
        showSong(current);
```

```
    }
}

void previousSong()
{
    if (current == NULL)
        cout << "Playlist is empty!\n";
    else if (current->prev == NULL)
        cout << "Start of Playlist!\n";
    else
    {
        current = current->prev;
        cout << "Now playing: ";
        showSong(current);
    }
}

int main()
{
    int choice;
    cout << "==============================\n";
    cout << "  MUSIC PLAYLIST MANAGEMENT\n";
    cout << "==============================\n";

    do
    {
        cout << "\n1. Add Song\n2. Delete Song\n3. Search Song\n"
             << "4. Display Playlist\n5. Next Song\n6. Previous Song\n"
             << "7. Exit\nEnter your choice: ";
        cin >> choice;

        switch (choice)
        {
            case 1: addSong(); break;
            case 2: deleteSong(); break;
            case 3: searchSong(); break;
            case 4: displayPlaylist(); break;
            case 5: nextSong(); break;
            case 6: previousSong(); break;
            case 7: cout << "Goodbye!\n"; break;
            default: cout << "Invalid choice! Try again.\n";
        }
```

```
    } while (choice != 7);

    return 0;
}
```

8. Sample Output

The menu is shown before every choice; it is abbreviated as `[menu]` after the first time.

```
==============================
  MUSIC PLAYLIST MANAGEMENT
==============================

1. Add Song
2. Delete Song
3. Search Song
4. Display Playlist
5. Next Song
6. Previous Song
7. Exit
Enter your choice: 1
Enter song name: Kesariya
Enter artist name: Arijit Singh
Enter duration (in seconds): 268
Song added!

[menu] 1
Enter song name: Tum Hi Ho
Enter artist name: Arijit Singh
Enter duration (in seconds): 262
Song added!

[menu] 4
1. Kesariya | Arijit Singh | 4:28
2. Tum Hi Ho | Arijit Singh | 4:22

[menu] 5
Now playing: Tum Hi Ho | Arijit Singh | 4:22

[menu] 6
Now playing: Kesariya | Arijit Singh | 4:28
```

```
[menu] 3
Enter song name to search: Tum Hi Ho
Song found: Tum Hi Ho | Arijit Singh | 4:22

[menu] 2
Enter song name to delete: Kesariya
Song deleted!

[menu] 4
1. Tum Hi Ho | Arijit Singh | 4:22

[menu] 9
Invalid choice! Try again.

[menu] 7
Goodbye!
```

9. Time Complexity

Operation	Complexity	Reason
Add song	O(1)	Uses the `tail` pointer, no traversal
Delete song	O(n)	Must search for the song by name
Search song	O(n)	Linear traversal from head
Display playlist	O(n)	Visits every node once
Next / Previous	O(1)	Follows one pointer from `current`

10. Conclusion

The Music Playlist Management System was implemented using a doubly linked list in C++. The program demonstrates dynamic memory allocation, pointer manipulation, and insertion, deletion, searching and traversal on a linked list. The `prev` and `next` pointers allow movement in both directions through the playlist, and the menu-driven `do-while` / `switch` structure keeps the program running until the user chooses Exit.